МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА» (БГТУ им. В.Г. Шухова)
Кафедра программного обеспечения
вычислительной техники и автоматизированных
систем

Лабораторная работа №7
по дисциплине: БД
тема: «**Организация взаимодействия с базой данных через приложение, использующее технологию ORM**»

Выполнил: студент группы ВТ-231
Масленников Д. А.
Проверили:
Панченко М. В.

Белгород 2025

Цель работы: разработать приложение, использующее технологию ORM, для взаимодействия с базой данных.

Задание к работе:

Вариант 14

1. Изучить технологию ORM.
2. Разработать снабженное графическим интерфейсом приложение, которое обеспечит подключение к базе данных, разработанной на основе предыдущих лабораторных работ, а также обеспечит выполнение запросов с использованием технологии ORM.

Структура моей базы данных из предыдущих работ

```
railway=# \dt
               Список отношений
 Схема  |      Имя       |   Тип    | Владелец
--------+----------------+----------+----------
 public | car            | таблица | postgres
 public | contactdetails | таблица | postgres
 public | passenger      | таблица | postgres
 public | route          | таблица | postgres
 public | schedule       | таблица | postgres
 public | seat           | таблица | postgres
 public | ticket         | таблица | postgres
 public | ticketstatus   | таблица | postgres
 public | train          | таблица | postgres
(9 строк)
```

Структура моего проекта

```
~/projects/university/5sem/db/lab7/railway_api on master ?1 > l
итого 48K
drwxr-xr-x 4 danchomas danchomas 4,0K ноя 19 13:50 .
drwxr-xr-x 3 danchomas danchomas 4,0K ноя 19 13:51 ..
-rw-r--r-- 1 danchomas danchomas   79 ноя 19 13:49 config.py
-rw-r--r-- 1 danchomas danchomas  382 ноя 19 13:41 database.py
-rw-r--r-- 1 danchomas danchomas  361 ноя 19 13:47 main.py
-rw-r--r-- 1 danchomas danchomas 1,9K ноя 19 13:41 models.py
drwxr-xr-x 2 danchomas danchomas 4,0K ноя 19 13:49 __pycache__
-rw-r--r-- 1 danchomas danchomas  295 ноя 19 13:30 requirements.txt
-rw-r--r-- 1 danchomas danchomas 4,7K ноя 19 13:49 routers.py
-rw-r--r-- 1 danchomas danchomas  737 ноя 19 13:49 schemas.py
drwxr-xr-x 5 danchomas danchomas 4,0K ноя 19 13:29 venv
~/projects/university/5sem/db/lab7/railway_api on master ?1 >
```

Для реализации данной лабораторной работы я использовал FastAPI
config.py

```
DATABASE_URL = "postgresql+asyncpg://postgres:postgres@localhost:5432/railway"
```

database.py

```
from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine
from sqlalchemy.orm import sessionmaker
from config import DATABASE_URL

engine = create_async_engine(DATABASE_URL, echo=False)
async_session = sessionmaker(engine, class_=AsyncSession,
expire_on_commit=False)


async def get_db() -> AsyncSession:
  async with async_session() as session:
    yield session
```

models.py

```
from sqlalchemy import Column, Integer, String, ForeignKey, DateTime, Numeric
from sqlalchemy.orm import relationship, declarative_base
```

```
Base = declarative_base()


class Train(Base):
  __tablename__ = "train"
  id = Column(Integer, primary_key=True)
  trainnumber = Column(String)
  traintype = Column(String)
  schedules = relationship("Schedule", back_populates="train")


class Route(Base):
  __tablename__ = "route"
  id = Column(Integer, primary_key=True)
  departurestation = Column(String)
  arrivalstation = Column(String)
  schedules = relationship("Schedule", back_populates="route")


class Schedule(Base):
  __tablename__ = "schedule"
  id = Column(Integer, primary_key=True)
  trainid = Column(Integer, ForeignKey("train.id"))
  routeid = Column(Integer, ForeignKey("route.id"))
  departuredatetime = Column(DateTime)
  train = relationship("Train", back_populates="schedules")
  route = relationship("Route", back_populates="schedules")
  cars = relationship("Car", back_populates="schedule")


class Car(Base):
  __tablename__ = "car"
  id = Column(Integer, primary_key=True)
  scheduleid = Column(Integer, ForeignKey("schedule.id"))
  cartype = Column(String)
  totalseats = Column(Integer)
  schedule = relationship("Schedule", back_populates="cars")
  seats = relationship("Seat", back_populates="car")


class Seat(Base):
  __tablename__ = "seat"
  id = Column(Integer, primary_key=True)
  carid = Column(Integer, ForeignKey("car.id"))
  car = relationship("Car", back_populates="seats")
  ticket = relationship("Ticket", back_populates="seat", uselist=False)
```

```python
class Ticket(Base):
  __tablename__ = "ticket"
  id = Column(Integer, primary_key=True)
  seatid = Column(Integer, ForeignKey("seat.id"))
  price = Column(Numeric(10, 2))
  seat = relationship("Seat", back_populates="ticket")
```

schemas.py

```python
from pydantic import BaseModel
from datetime import datetime


class TicketByTrainTypeSchema(BaseModel):
  train_type: str
  total_tickets: int


class AvgPriceByRouteSchema(BaseModel):
  departure_station: str
  arrival_station: str
  average_price: float


class SeatsByCarTypeSchema(BaseModel):
  car_type: str
  total_seats: int


class PopularRouteSeasonSchema(BaseModel):
  season: str
  departure_station: str
  arrival_station: str
  passengers: int


class TrainRatingByCarTypeSchema(BaseModel):
  train_number: str
  train_type: str
  car_type: str
  sold_tickets: int


class TrainFromAToBSchema(BaseModel):
  train_number: str
  train_type: str
  departure: datetime
```

```
  arrival: datetime
```

routers.py

```
from fastapi import APIRouter, Depends
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select, func, extract, case, text
from typing import List
from database import get_db
from models import Train, Route, Schedule, Car, Seat, Ticket
from schemas import (
  TicketByTrainTypeSchema,
  AvgPriceByRouteSchema,
  SeatsByCarTypeSchema,
  PopularRouteSeasonSchema,
  TrainRatingByCarTypeSchema,
  TrainFromAToBSchema,
)

router = APIRouter(prefix="/api", tags=["Железнодорожные запросы"])


@router.get("/tickets-by-train-type",
response_model=List[TicketByTrainTypeSchema])
async def tickets_by_train_type(db: AsyncSession = Depends(get_db)):
  query = (
    select(Train.traintype, func.count(Ticket.id).label("total_tickets"))
    .join(Schedule, Train.id == Schedule.trainid)
    .join(Car, Schedule.id == Car.scheduleid)
    .join(Seat, Car.id == Seat.carid)
    .join(Ticket, Seat.id == Ticket.seatid)
    .group_by(Train.traintype)
  )
  result = await db.execute(query)
  return [
    TicketByTrainTypeSchema(train_type=r[0], total_tickets=r[1])
    for r in result.all()
  ]


@router.get("/avg-price-by-route", response_model=List[AvgPriceByRouteSchema])
async def avg_price_by_route(db: AsyncSession = Depends(get_db)):
  query = (
    select(
      Route.departurestation,
      Route.arrivalstation,
```

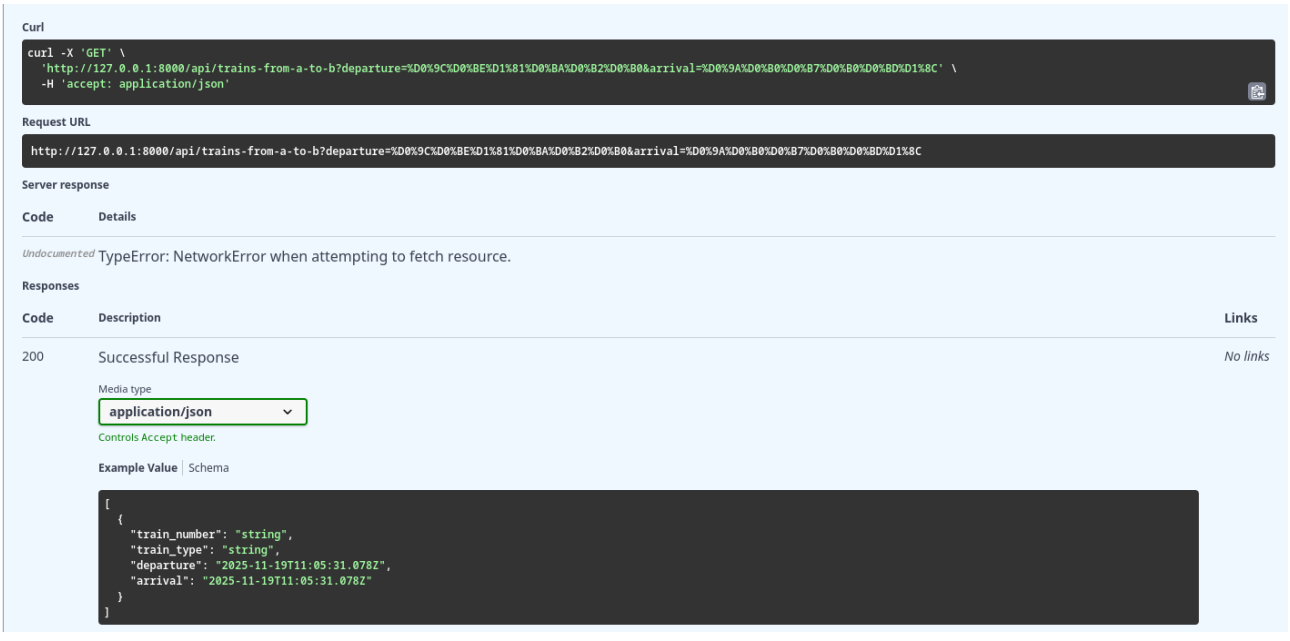
```
      func.round(func.avg(Ticket.price), 2).label("avg_price"),
    )
    .join(Schedule, Route.id == Schedule.routeid)
    .join(Car, Schedule.id == Car.scheduleid)
    .join(Seat, Car.id == Seat.carid)
    .join(Ticket, Seat.id == Ticket.seatid)
    .group_by(Route.departurestation, Route.arrivalstation)
  )
  result = await db.execute(query)
  return [
    AvgPriceByRouteSchema(
      departure_station=r[0], arrival_station=r[1], average_price=r[2]
    )
    for r in result.all()
  ]


@router.get("/seats-by-car-type", response_model=List[SeatsByCarTypeSchema])
async def seats_by_car_type(db: AsyncSession = Depends(get_db)):
  query = select(Car.cartype, func.sum(Car.totalseats)).group_by(Car.cartype)
  result = await db.execute(query)
  return [SeatsByCarTypeSchema(car_type=r[0], total_seats=r[1]) for r in
result.all()]


@router.get("/popular-routes-by-season",
response_model=List[PopularRouteSeasonSchema])
async def popular_routes_by_season(db: AsyncSession = Depends(get_db)):
  season = case(
    (extract("month", Schedule.departuredatetime).in_([12, 1, 2]), "зима"),
    (extract("month", Schedule.departuredatetime).in_([3, 4, 5]), "весна"),
    (extract("month", Schedule.departuredatetime).in_([6, 7, 8]), "лето"),
    else_="осень",
  ).label("season")

  query = (
    select(
      season, Route.departurestation, Route.arrivalstation,
func.count(Ticket.id)
    )
    .join(Seat, Ticket.seatid == Seat.id)
    .join(Car, Seat.carid == Car.id)
    .join(Schedule, Car.scheduleid == Schedule.id)
    .join(Route, Schedule.routeid == Route.id)
    .group_by(season, Route.departurestation, Route.arrivalstation)
    .order_by(season, func.count(Ticket.id).desc())
  )
```

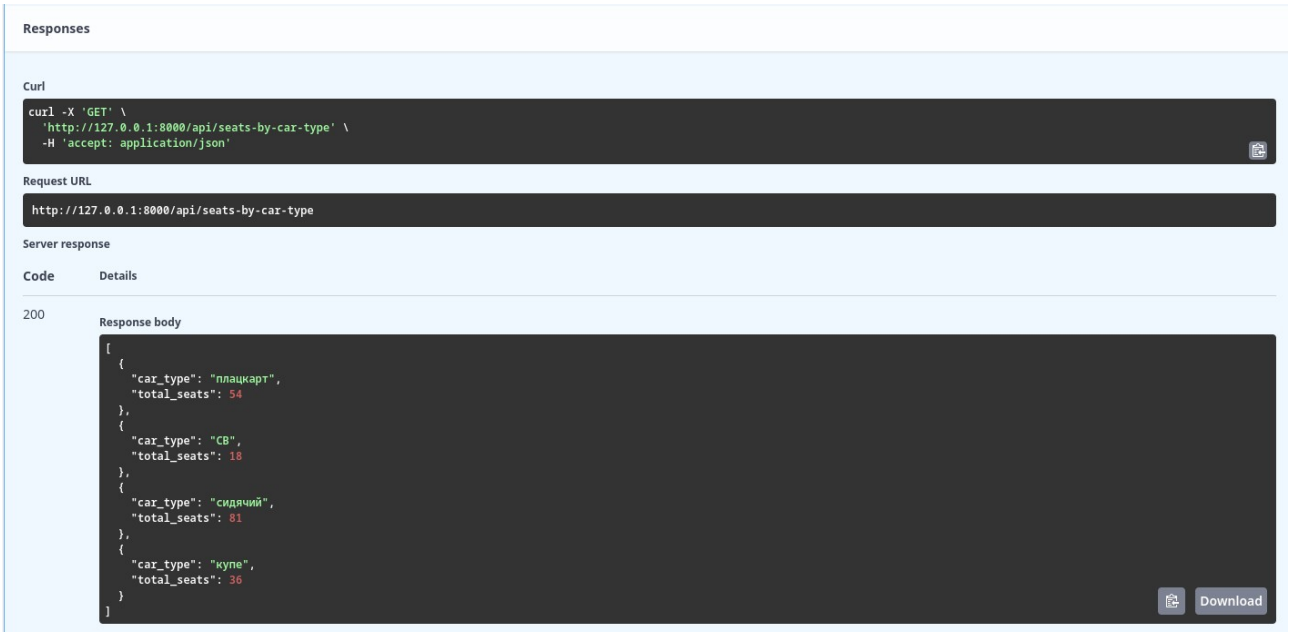
```
  result = await db.execute(query)
  return [
    PopularRouteSeasonSchema(
      season=r[0], departure_station=r[1], arrival_station=r[2], passengers=r[3]
    )
    for r in result.all()
  ]


@router.get(
  "/train-rating-by-car-type", response_model=List[TrainRatingByCarTypeSchema]
)
async def train_rating_by_car_type(db: AsyncSession = Depends(get_db)):
  query = (
    select(Train.trainnumber, Train.traintype, Car.cartype,
func.count(Ticket.id))
    .join(Schedule, Train.id == Schedule.trainid)
    .join(Car, Schedule.id == Car.scheduleid)
    .join(Seat, Car.id == Seat.carid)
    .join(Ticket, Seat.id == Ticket.seatid)
    .group_by(Train.trainnumber, Train.traintype, Car.cartype)
    .order_by(Car.cartype, func.count(Ticket.id).desc())
  )
  result = await db.execute(query)
  return [
    TrainRatingByCarTypeSchema(
      train_number=r[0], train_type=r[1], car_type=r[2], sold_tickets=r[3]
    )
    for r in result.all()
  ]


@router.get("/trains-from-a-to-b", response_model=List[TrainFromAToBSchema])
async def trains_from_a_to_b(
  departure: str, arrival: str, db: AsyncSession = Depends(get_db)
):
  query = text("SELECT * FROM get_trains_from_a_to_b(:dep, :arr)")
  result = await db.execute(query, {"dep": departure, "arr": arrival})
  rows = result.fetchall()
  return [
    TrainFromAToBSchema(
      train_number=row[0], train_type=row[1], departure=row[2], arrival=row[3]
    )
    for row in rows
  ]
```

main.py

```python
from fastapi import FastAPI
from routers import router
import uvicorn

app = FastAPI(
  title="railway api",
  description="lab 7 Maslennikov",
)

app.include_router(router)


@app.get("/")
async def root():
  return {"code": 200, "message": "успех"}


if __name__ == "__main__":
  uvicorn.run("main:app", host="127.0.0.1", port=8000, reload=True)
```

Результат работы:





Вывод: в ходе выполнения лабораторной работы были достигнуты цели лабораторной работы